

INTCK

Say exactly what you are counting

01 / THE SAS EXAMPLE

```
data test;
  input start_date :date9. end_date :date9.;
  days = intck('day', start_date, end_date);
  months = intck('month', start_date, end_date);
  years = intck('year', start_date, end_date);
  format start_date end_date date9.;
  datalines;
01JAN2023 15JAN2023
01JAN2023 01FEB2023
01JAN2023 31DEC2023
15MAR2023 20MAR2023
31JAN2023 01FEB2023
31DEC2023 01JAN2024
;
run;
proc print data=test noobs;
run;
```

REPORTED TEST SETUP / 21 SEP 2026

Claude Sonnet 4.5 / AWS Bedrock GovCloud
Claude Code 2.1.278 / effortLevel=high / prompt v0.1.0

ATTACH SOURCE OR PASTE IT AFTER THE PROMPT. START FRESH EACH TIME.

WHAT YOU WILL PRACTICE

Turn a familiar business rule into an explicit conversion requirement.

Default INTCK counts calendar boundaries. Think of turning a calendar page: January 31 to February 1 crosses a month boundary, although only one day passes. Compare all six rows.

EXPECTED RESULT

start_date	end_date	days	months	years
01JAN2023	15JAN2023	14	0	0
01JAN2023	01FEB2023	31	1	0
01JAN2023	31DEC2023	364	11	0
15MAR2023	20MAR2023	5	0	0
31JAN2023	01FEB2023	1	1	0
31DEC2023	01JAN2024	1	1	1

Last two rows test boundaries rather than elapsed periods.

REPORTED MATCHES / ALL THREE COLUMNS

Prompt A 3/3 | Prompt B 3/3

Reference values checked in Python; SAS execution pending.

Prompt A: leave choices open

High degrees of freedom / Notice the decisions the model makes for you.

COPY THIS PROMPT + THE SAS EXAMPLE

Convert this SAS code to Python.
Show code in this response. Do not create or run files, or read existing Python files.

REPORTED TRIAL RESULTS / 21 SEP 2026

Two methods, the same boundary counts

Run	Implementation	Values match
A1	Custom functions + .apply()	Yes
A2	Vectorized inline	Yes
A3	Vectorized inline	Yes

A1 used functions; A2/A3 used direct Series formulas.

All explained boundary counting; A runs had no assertions.

A shared result does not require identical code.

SOURCES: results-20260921.csv; trial-log-20260921.md; analysis-20260921.md

WHY CHOICES REMAIN OPEN

- 1

Same rule, different structure
A1 put the date arithmetic in functions. A2 and A3 wrote it inline. Both structures matched the six supplied rows.
- 2

The one-day rows test the meaning
January 31 to February 1 crosses a month. December 31 to January 1 crosses both a month and a year.
- 3

Compare behavior before style
Different code is not automatically a failed conversion. Check the three columns and input order before choosing a method.

ANTI-PATTERN: "MONTHS BETWEEN DATES"

Completed months and crossed month boundaries answer different questions. State the counting rule and include a one-day boundary example.

Dividing elapsed days by 30 does not count boundaries.

Prompt B: make the choices explicit

Low degrees of freedom / State the behavior, then ask for a check.

COPY THIS PROMPT + THE SAME SAS EXAMPLE

```
Convert this SAS code to Python using pandas.
Show code in this response. Do not create or run files, or read
existing Python files.

- Recreate the six rows in order; keep the same column names.
- Parse dates with pd.to_datetime(..., format='%d%b%Y').
  The inputs are valid dates with English month names.
- Preserve default discrete INTCK behavior: count boundaries.
- days: subtract start_date from end_date and use .dt.days.
- months: 12 * (end year - start year) + end month - start month.
- years: end year - start year.
- Return one complete script and include assertions for:
  days = [14, 31, 364, 5, 1, 1]
  months = [0, 1, 11, 0, 1, 1]
  years = [0, 0, 0, 0, 0, 1]
- Calculate from the input dates; do not hard-code result columns.
Briefly explain why a one-day interval can cross a month or year.
```

WHY THESE DETAILS HELP

- 1 Define the meaning**
Default discrete INTCK counts boundaries. Keep this exercise scoped to ordinary day, month and year date intervals.
- 2 Choose the route**
Specify pandas, date parsing and formulas. All three B runs report vectorized inline calculations.
- 3 Make success visible**
Assert every result column. B1-B3 report the expected lists and explain why one day can cross a month or year.

REPORTED RESULTS / THREE B RUNS

Run	Values match	Assertions
B1	Yes	Passed
B2	Yes	Passed
B3	Yes	Passed

CHECK IT YOURSELF
Check every row in days, months and years.
One day can cross one month and one year.

One behavior, three implementations

Count ordinary discrete day, month and year boundaries. A one-day interval can cross a month or year, and reversed dates produce signed counts.

SAS / SOURCE BEHAVIOR

```
%macro transform(in=, out=);
data &out;
  set &in;
  days = intck('day',
    start_date, end_date);
  months = intck('month',
    start_date, end_date);
  years = intck('year',
    start_date, end_date);
  format start_date end_date date9.;
run;
%mend;
```

INTCK defaults to discrete intervals. Loader reads DATE9. values; macro retains the SAS date columns. SAS harness is unexecuted.

PYTHON / PANDAS

```
def translate(df):
    out = df.copy(deep=True)
    out["start_date"] = parse_date9(
        out["start_date"])
    out["end_date"] = parse_date9(
        out["end_date"])
    s = out["start_date"].dt
    e = out["end_date"].dt
    out["days"] = (
        out["end_date"] - out["start_date"]
    ).dt.days
    out["years"] = e.year - s.year
    out["months"] = (
        12*out["years"] + e.month-s.month)
    return out
```

parse_date9 maps English month names before pd.to_datetime. The copied frame holds datetime columns; .dt supplies calendar components.

R / BASE

```
translate <- function(df) {
  out <- df
  out$start_date <- parse_date9(
    out$start_date)
  out$end_date <- parse_date9(out$end_date)
  s <- out$start_date
  e <- out$end_date
  out$days <- as.integer(e - s)
  out$years <- part(e,"%Y") - part(s,"%Y")
  out$months <- 12*out$years +
    part(e,"%m") - part(s,"%m")
  out
}
```

parse_date9 maps English month names to Date values. part(x, format) extracts integer calendar components; both helpers are in reference.R.

Calendar boundaries, not durations
Month counts come from year/month positions. Days divided by 30 and completed-anniversary calculations answer different questions.

Date representations differ
SAS dates, pandas datetimes and R Date values store dates differently. Compare calendar meaning and boundary counts, preserving row order.

Three checks that earn the pass

Use the same inputs and expected behavior in each language. Check the contract as well as the values.

T1 / Original fixture: all 18 counts

INPUT / CONDITION

Original six start/end pairs, including Jan 31 -> Feb 1 and Dec 31 -> Jan 1.

EXPECTED

days [14,31,364,5,1,1]; months [0,1,11,0,1,1]; years [0,0,0,0,0,1].

WHAT THIS CATCHES

Days/30 month estimates, elapsed-anniversary rules or incorrect year counting.

T2 / Added calendar boundary probes

INPUT / CONDITION

Same date; next day in same month; Feb 29 -> Mar 1; Dec 31 -> Jan 1; reversed year boundary.

EXPECTED

[day,month,year]: [0,0,0], [1,0,0], [1,1,0], [1,1,1], [-1,-1,-1].

WHAT THIS CATCHES

Off-by-one counts, leap-month mistakes, loss of sign and fixed-length month approximations.

T3 / Preserve dates, types and order

INPUT / CONDITION

All CSV pairs in reverse order with custom row identifiers.

EXPECTED

Inputs untouched; output dates retain calendar meaning; counts numeric; row order and all expected results preserved.

WHAT THIS CATCHES

Date truncation, input mutation, sorting, text result columns or position-dependent answers.

REFERENCE CODE + TESTS: [module-06-intck/appendix/](#)

Run: `python tools/run_appendix_tests.py --modules 06`

New reference code, separate from historical trials. SAS checks await execution; scope is stated in the companion README.